TECHNICAL REPORT

NOVEMBER 2026

GPU Performance Modeling and Evidence-Guided Optimization

Performance prediction, Triton kernel autotuning, and profiler-informed validation

GPU SYSTEMS / MACHINE LEARNING / PERFORMANCE ENGINEERING

Towsif Hassan

B.S. Computer Science · Texas State University

github.com/cyber-kuro/gpu-deep-learning-research

Contents

1. Executive summary 3
2. Research questions and study design 4
3. Experimental environment 5
4. Milestone 3: FP32 benchmarking and profiling 6
5. Stage A–B: Dataset preparation and latency prediction 8
6. Stage B: Data efficiency and MLP behavior 10
7. Stage C: Triton configuration dataset and autotuning policies 11
8. Stage D: Nsight A/B case study and evidence-guided decision 14
9. Limitations, conclusions, and future work 16
10. Reproducibility and artifact index 17
11. References 18

This report distinguishes **measured execution results**, **offline evaluation**, and **diagnostic interpretation**. Claims are limited to the experiments actually performed.

1. Executive summary

This project studies three connected questions in GPU performance optimization: (i) whether workload shape can predict matrix-multiplication latency, (ii) whether a learned model can recommend low-latency Triton kernel configurations from a restricted candidate set, and (iii) whether hardware profiling and fresh timing measurements can validate an optimization decision when model predictions and profiler heuristics disagree.

0.010719 ms	3.12%	1.119×
BEST HELD-OUT RUNTIME-PREDICTION MAE	MLP AUTOTUNING MEAN REGRET ON HELD-OUT SHAPES	VERIFIED SPEEDUP IN THE A/B CASE STUDY

Main findings

- **PyTorch FP32 latency prediction.** Among the tested predictors, a Random Forest achieved the best held-out accuracy on 18 matrix shapes, with **0.010719 ms MAE** and **11.56% mean relative error**.
- **Data efficiency.** For the small MLP predictor, mean validation MAE improved substantially as training data increased from 8 to 42 workload shapes and then changed only marginally from 42 to 84 shapes in this experiment.
- **Restricted Triton autotuning.** On a four-configuration candidate set, the MLP policy achieved **3.12% mean regret** on 18 held-out shapes, compared with **9.19%** for a fixed configuration selected from training data.
- **Evidence-guided validation.** In a focused A/B comparison, fresh paired CUDA Graph timing showed configuration B outperforming configuration A in all five rounds, despite lower achieved occupancy and an incorrect model preference for A.

Interpretation. Latency prediction, configuration selection, and optimization validation are related but distinct tasks. A model with lower regression error is not automatically the best decision policy, and a profiler metric such as occupancy cannot be treated as a direct predictor of end-to-end latency.

2. Research questions and study design

Research question	Concrete experiment	Primary evaluation
RQ1. Can workload shape predict PyTorch GEMM latency?	Constant baseline, log-linear Ridge, Random Forest, and MLP on disjoint FP32 workload splits.	Held-out MAE and mean relative error.
RQ2. How sensitive is an MLP to limited data?	Repeated training with 8, 21, 42, and 84 training shapes across three seeds.	Mean and standard deviation of validation MAE.
RQ3. Can learned recommendations reduce configuration regret?	FP16 Triton dataset with four candidate configurations; RF and MLP recommendation policies compared against fixed, random-budget, and exhaustive-oracle baselines.	Mean selection regret and within-5% frequency.
RQ4. Can diagnostics and execution tests overturn a wrong model preference?	Nsight Compute A/B profiling, numerical correctness checks, and five paired CUDA Graph timing rounds.	Evidence-supported accept/reject decision.

The study progresses in stages: fundamental setup and correctness, FP32 benchmarking, workload-level modeling, Triton configuration modeling, and a final profiler-guided decision case study.

3. Experimental environment

GPU	NVIDIA GeForce RTX 5090, compute capability 12.0
CPU	AMD Ryzen 7 7800X3D
System memory	32 GB DDR5
Operating environment	Windows 11 with WSL2 Ubuntu
PyTorch	2.15.0+cu132
CUDA build	13.2
Triton	3.9.0
Profiler	NVIDIA Nsight Compute 2026.4.0
Precision settings	FP32 for the PyTorch matrix-multiplication study; FP16 for the Triton autotuning study

> The report aggregates results already generated in the project workspace. Figures are reconstructed from saved outputs and recorded summaries; the report is a synthesis of completed experiments rather than a fresh end-to-end rerun of every script inside this report-generation environment.

4. Milestone 3: FP32 benchmarking and profiling

Following standard PyTorch profiling and CUDA benchmarking practice [2, 4], the initial benchmarking study measured square matrix multiplication performance for matrix sizes from 256 to 4096. Each shape was warmed up, timed repeatedly with CUDA events, summarized by median latency, and converted to achieved throughput in TFLOP/s. A PyTorch profiler run was used to separate CPU-side dispatch from GPU-side execution.

Matrix size	Median latency (ms)	Achieved throughput (TFLOP/s)
256 × 256	0.0322	1.04
512 × 512	0.0286	9.37
1024 × 1024	0.0666	32.23
2048 × 2048	0.3294	52.16
4096 × 4096	2.0200	68.04

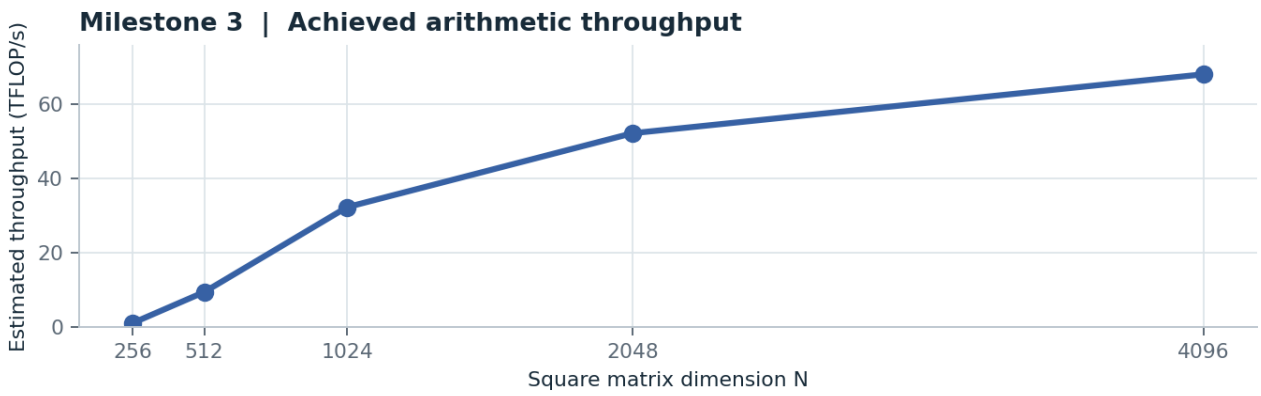


Figure 1. Throughput increases with matrix size as larger workloads amortize fixed overhead more effectively.

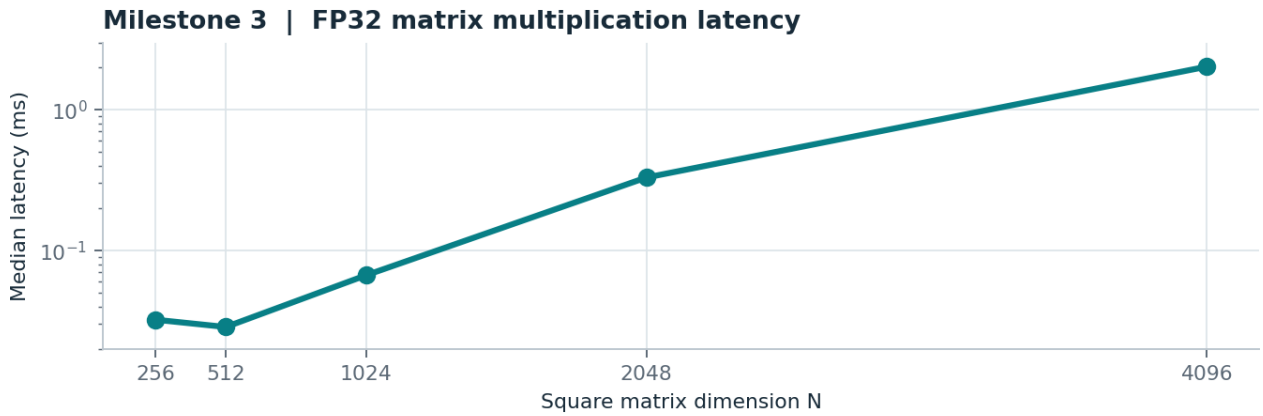


Figure 2. Latency also increases with matrix size, illustrating that higher throughput does not imply lower runtime.

This stage established the basic benchmarking workflow used later: explicit warmup, synchronized timing, repeated measurements, summary statistics, and clear separation between measured GPU work and CPU-side orchestration.

5. Stage A–B: Dataset preparation and latency prediction

A reproducible set of **120 unique workload shapes** was sampled, each with **five trial-batch medians**, producing **600 raw FP32 latency records**. Shapes were split by unique workload into 84 training, 18 validation, and 18 held-out test workloads. A quality audit retained high-variability workloads instead of discarding them, since the research aim was to model realistic performance behavior rather than only the easiest cases.

Held-out latency prediction results

Method	Test MAE (ms)	Mean relative error
Constant baseline	0.027749	30.15%
Log-linear Ridge	0.013174	16.57%
Random Forest	**0.010719**	**11.56%**
MLP	0.017672	32.02%

The Random Forest was the strongest predictor on this dataset and split. This does not establish a universal ordering across all kernels or hardware platforms.

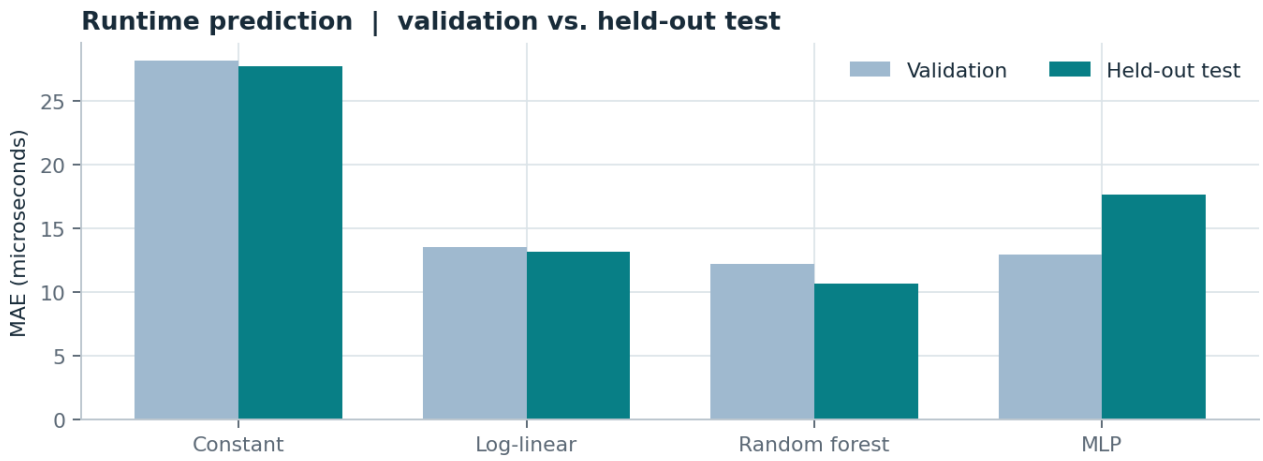


Figure 3. Comparison of held-out model MAE values for the workload-level runtime-prediction task.

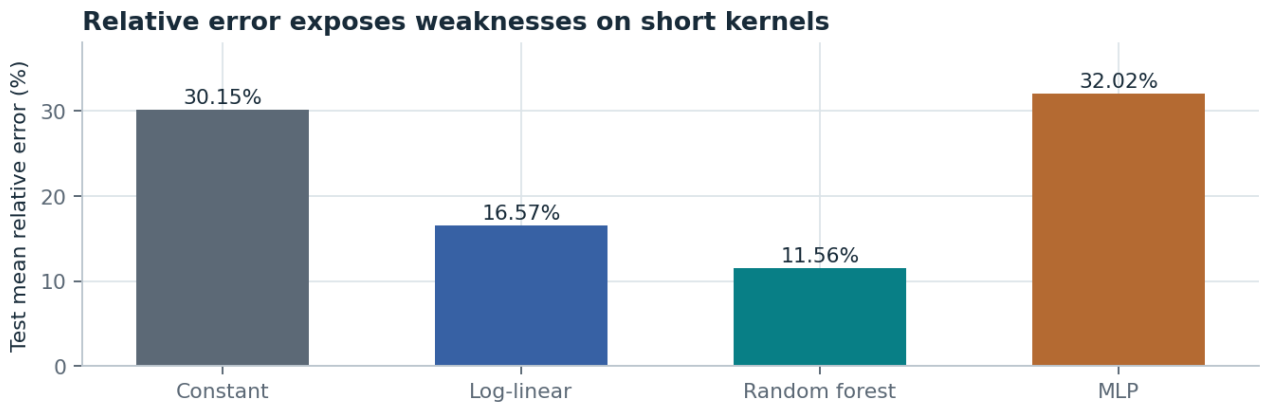


Figure 4. Mean relative error highlights that lower absolute MAE does not always imply uniformly good percentage accuracy across shapes.

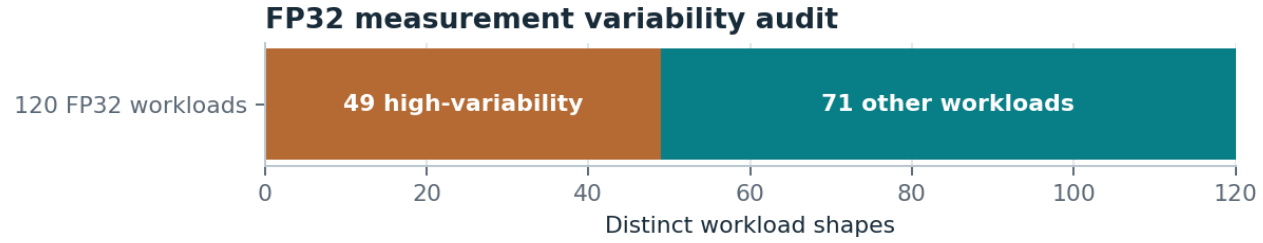


Figure 5. Workload variability audit. A substantial subset of shapes exhibited more than 10% coefficient of variation, motivating careful interpretation of close runtime differences.

6. Stage B: Data efficiency and MLP behavior

To test whether useful prediction accuracy could be achieved with limited data, the MLP was retrained using 8, 21, 42, and 84 training shapes across three random seeds. Validation performance improved quickly with additional data and then flattened between 42 and 84 training shapes in this specific study.

Training shapes	Mean validation MAE (ms)	Standard deviation (ms)
8	0.040945	0.009177
21	0.024434	0.013588
42	0.011739	0.003029
84	0.011672	0.002624

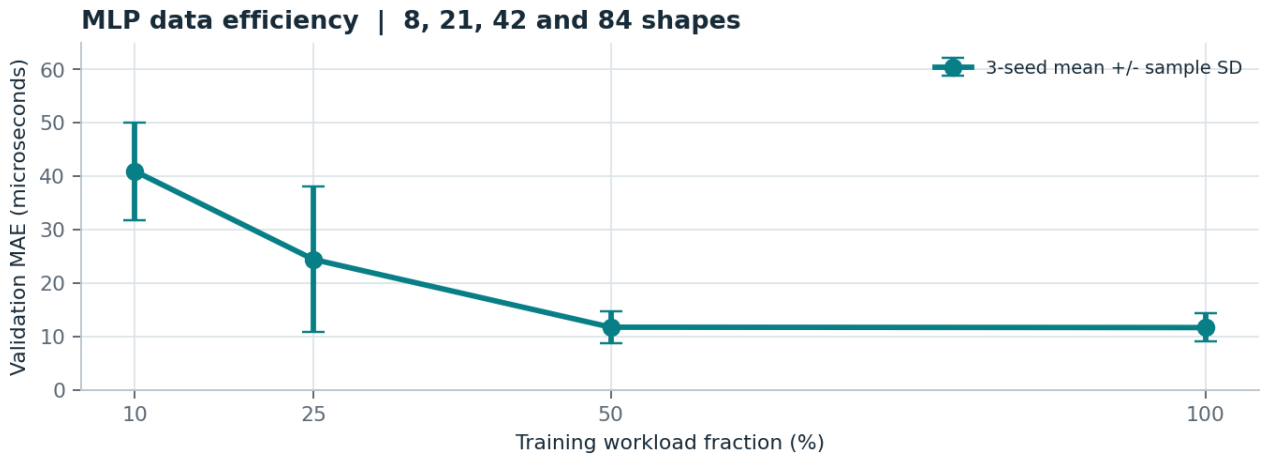


Figure 6. Validation MAE versus training-data fraction for the MLP across three random seeds.

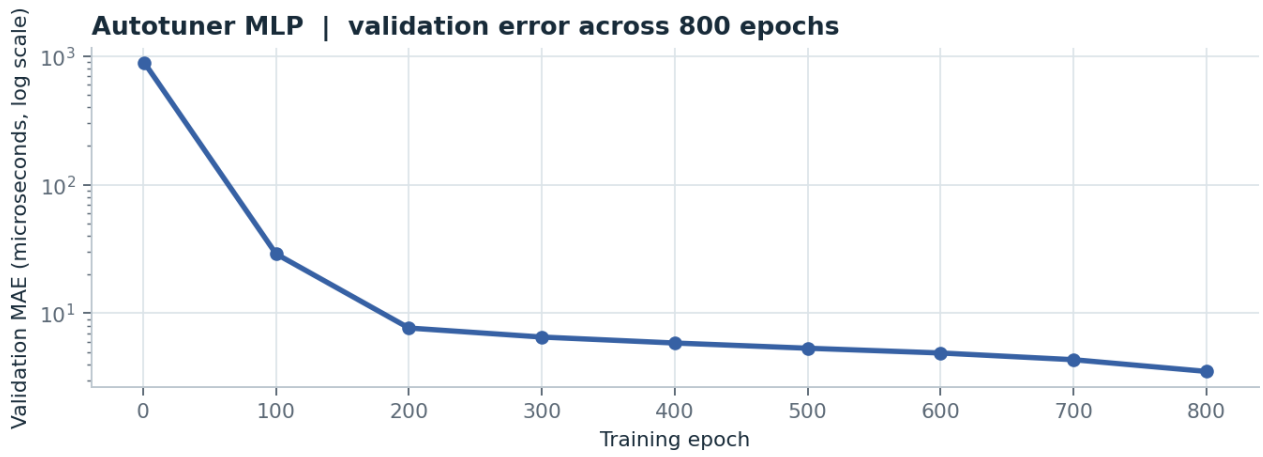


Figure 7. Example MLP training behavior showing the reduction in loss and validation error over epochs.

7. Stage C: Triton configuration dataset and autotuning policies

The Triton study moved from workload-level prediction to *configuration selection*, using configurable tiled matrix multiplication as described in the official Triton tutorial [3]. Four candidate tiled matrix-multiplication configurations were tested for each FP16 workload shape:

- A = `(32, 64, 32, 4)`
- B = `(64, 64, 32, 4)`
- C = `(64, 128, 32, 4)`
- D = `(64, 128, 64, 8)`

CUDA Graph replay timing was used to reduce host launch overhead for short GPU kernels [5]. Across 120 shapes, four configurations, and three trials, the study produced **1,440 raw measurements**, which were aggregated into **480 shape–configuration pairs**. Those pairs were split into 336 training, 72 validation, and 72 test examples with no workload-shape overlap between splits.

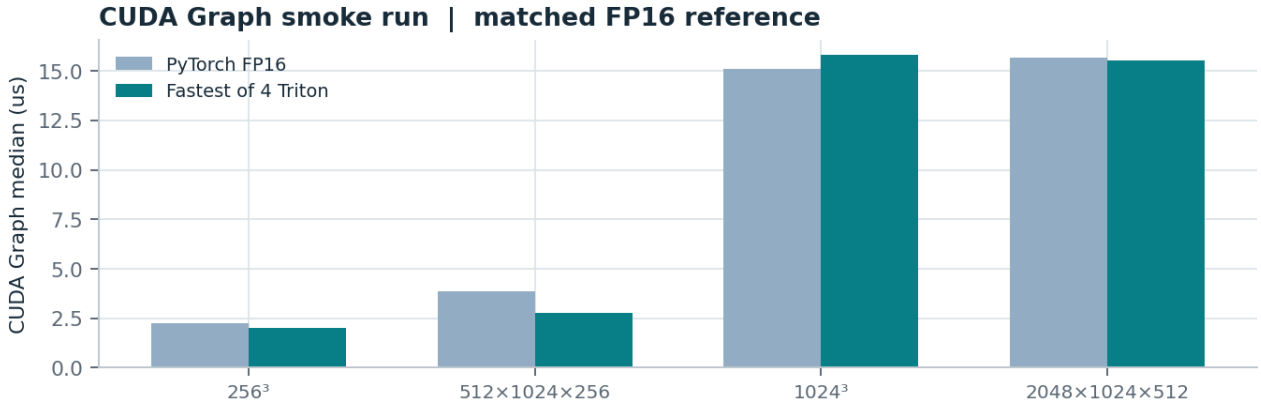


Figure 8. CUDA Graph smoke-test benchmark summary confirming the timing pipeline for the restricted Triton candidate set.

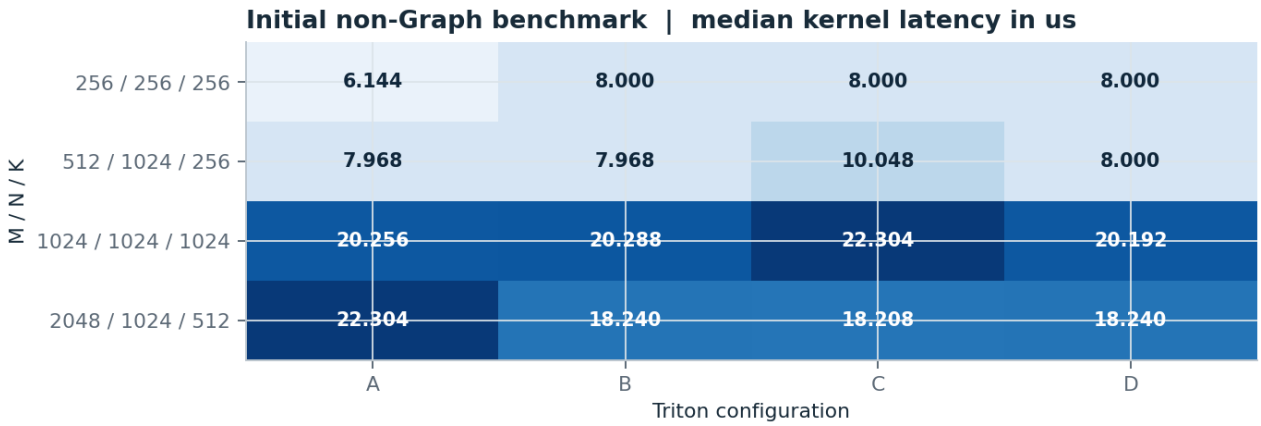


Figure 9. Overview of relative performance behavior across candidate configurations and workload shapes.

Held-out offline selection results

Strategy	Premeasured candidates per decision	Mean regret	Within 5% of best
MLP recommendation	**1**	**3.12%**	**83.3%**
Random Forest recommendation	1	3.95%	72.2%
Training-selected fixed configuration	1	9.19%	66.7%
Random search, 1 candidate	1	45.29%	40.6%
Random search, 2 candidates	2	19.34%	66.9%
Random search, 3 candidates	3	5.28%	84.7%
Exhaustive oracle	4	0.00%	100.0%

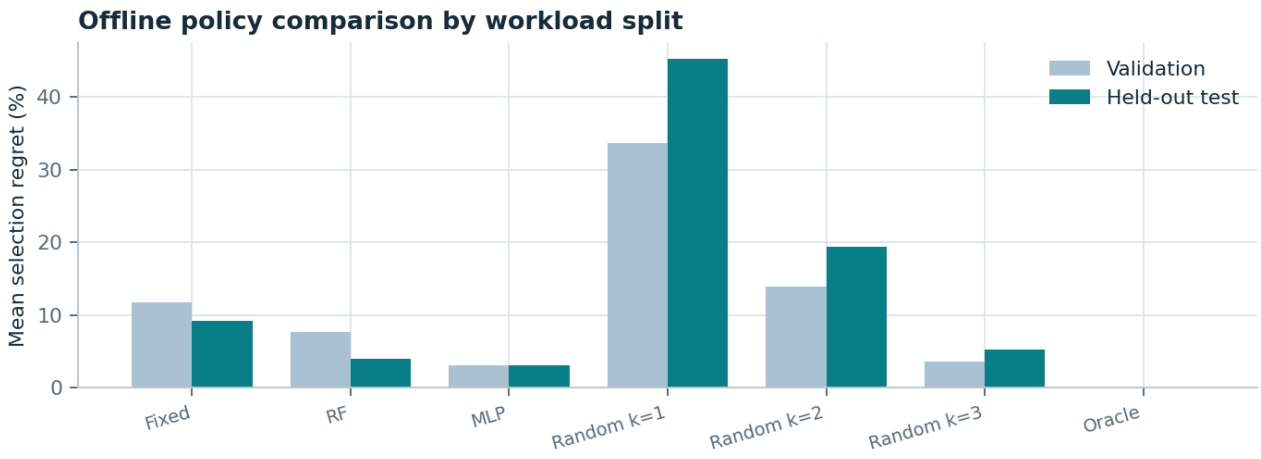


Figure 10. Offline policy comparison on held-out workload shapes.

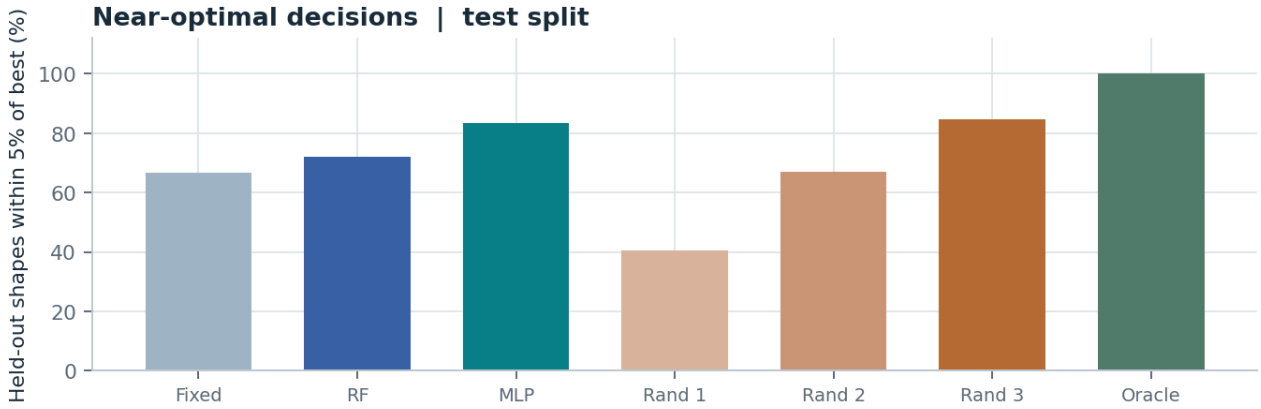


Figure 11. Fraction of held-out shapes for which each policy selected a configuration within 5% of the measured best.



Figure 12. Regret comparison between the Random Forest and MLP recommendation policies.

These are **offline** evaluations: all candidate latencies were already measured. The results indicate selection quality under a fixed candidate set, not a demonstrated reduction in full online tuning time.

8. Stage D: Nsight A/B case study and evidence-guided decision

Motivated by the diagnostic-to-transformation-to-validation workflow described in OPTIMAS [1], the final stage tested whether multiple evidence sources could support a transparent optimization decision on the workload `(512, 1024, 256)`. Two candidate configurations—A and B—were profiled with Nsight Compute [4], compared with frozen MLP predictions, checked for correctness against PyTorch across three random seeds, and then timed with five fresh paired CUDA Graph rounds.

Metric	A (32, 64, 32, 4)	B (64, 64, 32, 4)
Grid size	256	128
Registers per thread	54	86
Dynamic shared memory per block	12.29 KB	16.38 KB
Achieved occupancy	12.51%	8.31%
MLP predicted latency	4.144 µs	4.401 µs
Fresh CUDA Graph median latency	3.098 µs	**2.768 µs**
Nsight profiled duration	4.74 µs	4.74 µs

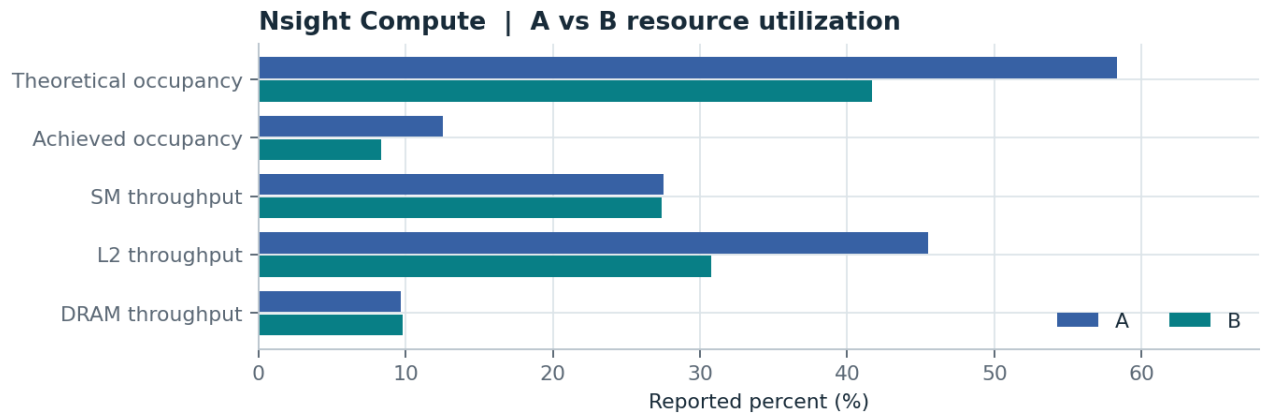


Figure 13. Nsight Compute comparison of occupancy-related metrics and resource use for configurations A and B.

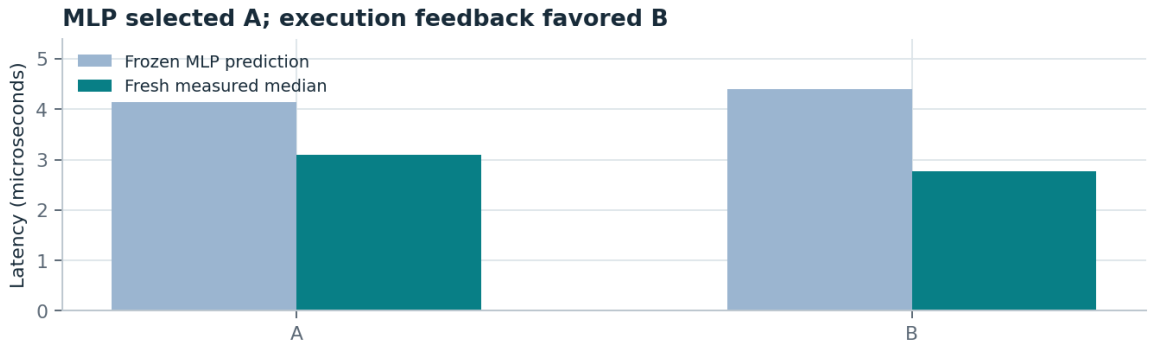


Figure 14. The frozen MLP predicted configuration A to be faster, but the measured outcome contradicted that recommendation.

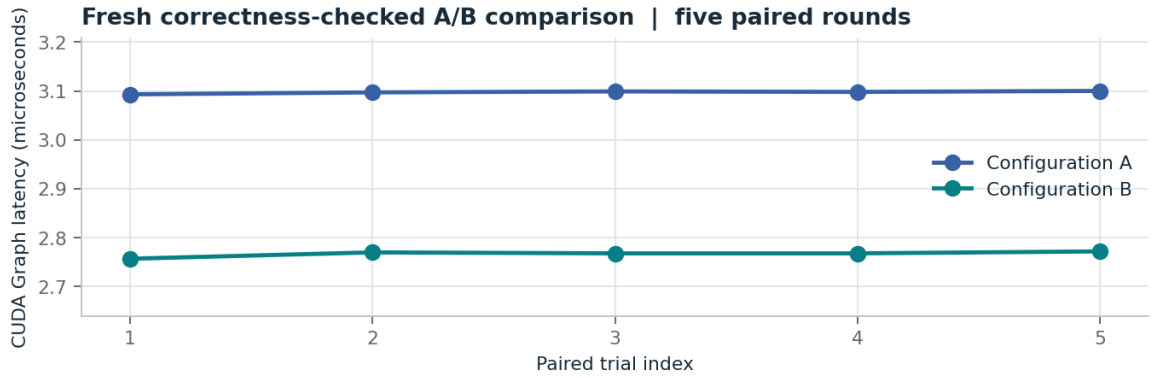


Figure 15. Paired fresh CUDA Graph timings. Configuration B was faster in all five rounds and achieved a median 1.119× speedup over A.

Decision rule. B was accepted when its median latency was at least 2% lower and it was faster in at least four of five paired rounds. Under that rule, the evidence-supported decision was **accept B**.

Relationship to OPTIMAS

OPTIMAS: An Intelligent Analytics-Informed Generative AI Framework for Performance Optimization, by Zaeed, Islam, and Indic [1], describes a modular AI-driven framework for translating performance diagnostics into candidate code transformations and verifying correctness and performance. This case study adopts the **diagnostic evidence → proposed change → correctness check → fresh benchmark → accept/reject** pattern.

The scope is intentionally narrower: the A-to-B tile change was predefined, not proposed autonomously by an LLM agent, and this implementation is not a reproduction of the multi-agent OPTIMAS system. It illustrates why execution feedback is critical when a model's preferred configuration is slower under fresh measurements.

Nsight Compute can instrument and replay kernels; its profiled duration should not be equated with uninstrumented CUDA Graph timing [4].

9. Limitations, conclusions, and future work

Limitations

- All experiments were conducted on one GPU and one kernel family; there is no cross-architecture validation.
- The Triton candidate set contains only four configurations; the exhaustive oracle is therefore the best of four, not the global best across all possible schedules.
- Held-out evaluations use only 18 workload shapes, so close performance ties can influence results.
- The runtime-prediction study and the autotuning study use different precisions and timing conditions.
- Offline policy evaluation does not include inference time, compilation cost, profiling cost, or the full end-to-end overhead of an online tuner.
- The final case study provides evidence-supported comparison, but not a complete causal explanation of why configuration B is faster.

Conclusions

This project demonstrates a coherent research pipeline from reproducible GPU benchmarking to machine-learning-based performance modeling and finally to evidence-guided optimization validation. The results support three main conclusions:

1. Runtime prediction from workload shape is feasible for this restricted matrix-multiplication setting, with the Random Forest providing the strongest held-out performance.
2. Learned configuration selection can substantially outperform a fixed policy under a restricted Triton candidate set, while still remaining meaningfully different from exhaustive search.
3. Fresh timing and correctness checks are essential, because neither a predictive model nor a single profiler metric should be trusted as a final decision signal.

Future work

- Expand the search space to more Triton configurations and larger workload diversity.
- Train directly for ranking or regret-aware objectives instead of only latency regression.
- Quantify uncertainty and close-tie sensitivity in configuration selection.
- Measure true online tuning cost and break-even reuse scenarios.
- Test transferability across other kernels and GPU architectures.

10. Reproducibility and artifact index

The project repository includes the scripts, datasets, and result files used throughout the study. Core artifacts include:

Category	Artifacts
Milestone 3 benchmarking	`benchmark_matmul.py`, `summary_result.py`, `plot_results.py`, `matmul_results.csv`, `matmul_summary.csv`
Workload prediction	`collect_data.py`, `prepare_dataset.py`, `train_baselines.py`, `train_mlp.py`, `final_evaluation.py`, `gpu_performance_data.csv`
Data efficiency	`data_efficiency.py`, `results/data_efficiency.csv`, `results/data_efficiency_summary.csv`
Triton autotuning	`triton_matmul.py`, `benchmark_triton.py`, `prepare_autotuning.py`, `train_autotuner.py`, `train_autotuner_mlp.py`, `final_autotuning_evaluation.py`, `triton_autotuning_data.csv`
Diagnostic case study	`profile_triton.py`, `evidence_guided_optimizer.py`, `results/profile_A.ncu-repz`, `results/profile_B.ncu-repz`, `results/evidence_guided_report.json`, `results/evidence_guided_timings.csv`
Project documentation	`README.md`, `milestone-04-performance-ml/README.md`, `docs/research_report.md`, `docs/optimization_case_study.md`

Note: some large binary outputs (for example, model checkpoints and profiler reports) may be omitted from a public repository snapshot depending on version-control decisions. Reproducibility in that case should rely on scripts and documented procedures to regenerate those artifacts.

11. References

References document related work and measurement methodology. All numerical results in the report are from the project's recorded experiments; they are not attributed to these sources.

1. M. Zaeed, T. Z. Islam, and V. Indic, *OPTIMAS: An Intelligent Analytics-Informed Generative AI Framework for Performance Optimization*, arXiv:2604.23892 (2026). https://arxiv.org/abs/2604.23892

2. PyTorch Contributors, *PyTorch Profiler*, PyTorch Documentation. https://docs.pytorch.org/docs/stable/profiler.html (accessed November 2026).

3. Triton Contributors, *Matrix Multiplication*, Triton Documentation. https://triton-lang.org/main/getting-started/tutorials/03-matrix-multiplication.html (accessed November 2026).

4. NVIDIA, *Nsight Compute: Profiling Guide*, NVIDIA Developer Documentation. https://docs.nvidia.com/nsight-compute/ProfilingGuide/ (accessed November 2026).

5. Triton Contributors, *triton.testing.do_bench_cudagraph*, Triton Python API. https://triton-lang.org/main/python-api/generated/triton.testing.do_bench_cudagraph.html (accessed November 2026).

6. T. Hassan, *GPU-Accelerated Deep Learning Research*, project repository (2026). https://github.com/cyber-kuro/gpu-deep-learning-research.